

Stragy_Fllow_Truth 策略说明

Stragy_Fllow_Truth.py 是一个面向 Polymarket 二元事件的简化事实跟随策略。新版策略不再使用复杂的 TP/CORE/HEDGE 规则树，而是改成更直观的“公平价格 + 边际 + 风控状态”模型。

核心思想

用户先回答一个最重要的问题：

我认为事实方向 FactSide 的真实概率是多少？

这个概率就是 fair_price 。策略随后用市场价格和 fair_price 比较：

- 当 $\text{fair_price} - \text{ask} \geq \text{entry_edge}$ ，说明买入事实方向有足够安全边际，可以开仓或加仓。
- 当 $\text{fair_price} - \text{bid} \leq \text{exit_edge}$ ，说明持仓边际已经消失，应该退出。
- 当触发止损、止盈、临近到期降风险、强制清仓时，风控优先于开仓逻辑。

Params

Params 是策略的结构性配置，通常写在策略代码里，并由 UI 提供默认值和覆盖能力。

参数	默认值	说明
FactSide	Yes	你认为最终会成真的方向，可选 Yes / No 。
fair_price	0.65	你对 FactSide 的公平概率估计。
entry_edge	0.05	开仓边际， fair_price - ask 达到该值才买入。
exit_edge	0.01	离场边际， fair_price - bid 低于该值则退出。
max_position_pct	0.5	最大目标仓位比例。
stop_loss_pct	0.25	按入场价或持仓均价计算的止损比例。
take_profit_pct	0.4	按入场价或持仓均价计算的止盈比例。
de_risk_days	1.0	距离结束不足该天数时线性降低目标仓位。
cooldown_seconds	600	调仓后冷却时间，冷却期内不加仓。

参数	默认值	说明
min_target_delta	0.03	普通调仓最小仓位差，小于该值不发动作。

Controls

Controls 是运行中的人工控制项，也可以理解为用户可随时修改的 UserState 。它们有默认值，不需要用户每次手动设置。

控制项	默认值	说明
manual_pause_open	False	暂停开新仓或加仓，但不阻止止盈、止损和平仓。
force_flat	False	强制把事实方向和反方向目标仓位都设为 0。
risk_scale	1.0	临时缩放目标仓位，范围 0-1 。
debug_raw_inputs	False	在 FunctionJson.print 输出原始参数和 UseData 字段，便于排查。

RuntimeState

RuntimeState 是策略运行后自动维护的状态，不建议用户频繁手动改。它也有默认值。

状态	默认值	说明
last_signal	none	最近一次决策标签。
last_target	0.0	最近一次事实方向目标仓位。
last_action_at	None	最近一次发出 SETPOS 的时间。
entry_price	None	最近一次开仓或加仓记录的入场价。
entry_side	None	entry_price 对应的方向。
cooldown_until	None	冷却结束时间。

决策流程

1. 读取 Params 、 Controls 、 RuntimeState 和市场数据。

2. 找到事实方向 FactSide 与反方向。
3. 读取事实方向的 ask、bid、当前仓位、均价、剩余天数。
4. 如果 force_flat=True，直接平仓。
5. 如果报价无效，保持当前仓位。
6. 如果已有仓位，先判断止损、止盈、边际消失。
7. 如果无仓位，只有 fair_price - ask >= entry_edge 才开仓。
8. 如果接近到期，按 day_to_end / de_risk_days 线性降低目标仓位。
9. 如果 manual_pause_open=True 或仍在冷却期，不允许新开仓或加仓。
10. 目标仓位变化达到 min_target_delta，或触发保护性退出时，输出 SETPOS。

输出格式

策略输出仍兼容现有运行系统：

```
{
  "schema_version": "2.0",
  "actions": [
    {
      "type": "SETPOS",
      "side": "Yes",
      "target_pct": 0.5,
      "leg": 0,
      "desc": "Set Yes target to 0.5000 (entry)."
    }
  ],
  "metrics": {},
  "print": [],
  "wake_reason": null,
  "state_updates": {}
}
```

UseData 字段兼容

策略会尽量兼容不同命名风格。以下字段都可以被识别：

- 价格：Yes_now_ask、Yes_AskPrice、L0_Yes_AskPrice、Yes_ask
- 买价：Yes_now_bid、Yes_BidPrice、L0_Yes_BidPrice、Yes_bid
- 仓位：Yes_Now_Pos、Yes_Pos、Yes_position_pct
- 数量：Yes_now_Qty、Yes_Qty

- 均价： `Yes_now_avgPrice` 、 `Yes_AvgPrice`
- 剩余时间： `day_to_end` 、 `days_to_end`

No 方向同理。

从旧版迁移

旧版策略里的 `FactSide_ref_ask` 会被兼容映射为新版 `fair_price`。旧版的 `risk` 、 `hedge_sell_bias` 、 `trend_sell_bias` 、 `core_sell_bias` 、 `start_day` 规则已经被移除，因为它们让策略含义变得不直观。

新版建议只保留一条主线：

我认为 Yes 的公平价格是 **0.65**。

如果市场 `ask` $\leq$ **0.60**，我愿意买。

如果市场 `bid` 已经接近或高于我的公平价格，或者亏损/盈利达到阈值，我退出。

设计建议

- `Params` 负责长期策略假设，例如公平价格、入场边际、最大仓位。
- `Controls/UserState` 负责短期人工干预，例如暂停开仓、强制清仓、临时降风险。
- `RuntimeState` 负责机器自动保存的运行记忆，例如入场价、冷却结束时间、最近信号。

这三类数据都应该有默认值。用户只需要修改真正想改的部分，不需要每次启动策略都填一堆初始状态。